Documentação Técnica

Ruby on Rails • PostgreSQL • Docker • TailwindCSS

Versão 1.0 — 2026

Desafio Técnico Nº 0004/2026 — Dunnas

1. Visão Geral do Sistema

O Capp (Condomínio App) é um sistema web de gerenciamento de chamados para condomínios, desenvolvido em Ruby on Rails como solução para o Desafio Técnico Nº 0004/2026. Ele permite que moradores registrem chamados de manutenção, segurança, limpeza e outros tipos, enquanto colaboradores os gerenciam e administradores controlam toda a estrutura do condomínio.

1.1 Objetivo

Fornecer uma plataforma centralizada e auditável para o registro, acompanhamento e resolução de chamados em condomínios, com controle de acesso por papéis (RBAC), histórico completo de interações e arquivamento automático de chamados antigos.

1.2 Funcionalidades Principais

- Cadastro de blocos e geração automática de unidades (apartamentos)
- Vinculação de moradores a unidades pelo administrador
- Sistema de chamados com tipos, status, SLA e arquivamento automático
- Três perfis de usuário: Administrador, Colaborador e Morador
- Upload de anexos nos chamados via Active Storage
- Comentários e histórico de interações por chamado
- Filtros avançados de chamados por status, tipo, bloco e período
- Vinculação de colaboradores a tipos de chamado específicos
- Log de auditoria completo de todas as ações do sistema
- Dashboard personalizado por perfil de usuário
- Interface responsiva com tema escuro/claro e menu sanduíche em mobile
- Arquivamento automático de chamados concluídos há mais de 30 dias (job agendado)

2. Stack de Tecnologias

Categoria	Tecnologia	Versão / Detalhe
Linguagem	Ruby	Linguagem principal do sistema
Framework Web	Ruby on Rails	~> 8.1.3 — padrão MVC
Banco de Dados	PostgreSQL	Relacional, rodando em container Docker
Asset Pipeline	Propshaft	Substituto moderno do Sprockets
Frontend	Tailwind CSS	~> 4.4 — utility-first, tema escuro
JS / Realtime	Turbo Rails + Stimulus	Hotwire stack para interatividade
Autenticação	Devise	Login, logout, recuperação de senha, BCrypt
Upload de Arquivos	Active Storage	Nativo do Rails; suporte a Cloudflare R2
Armazenamento S3	Cloudflare R2	Storage de arquivos em produção
Cache / Queue	Solid Cache + Solid Queue	Cache/Queue: *Solid Cache* + *Solid Queue* + *Solid Cable*
Conteinerização	Docker + Docker Compose	Container da aplicação Rails + PostgreSQL
Deploy	Render + Cloudflare	Hospedagem da aplicação + DNS
Controle de Acesso	RBAC manual	Implementado do zero (sem CanCanCan)
Versionamento BD	Rails Migrations	Histórico completo de alterações
Testes	Minitest + Capybara	TDD aplicado em todas as etapas
Análise de Segurança	Brakeman + Bundler Audit	Análise estática de vulnerabilidades
Linting	RuboCop Rails Omakase	Estilo de código Rails padrão
Localização	pt-BR	Sistema configurado em português do Brasil
Fuso Horário	Brasilia	America/Sao_Paulo (UTC-3)

3. Arquitetura do Sistema

3.1 Padrão MVC (Model-View-Controller)

O sistema segue o padrão MVC do Ruby on Rails:

- Model — Regras de negócio, validações, callbacks e acesso ao banco via ActiveRecord
- Controller — Processa requisições HTTP, aplica autenticação/autorização e coordena a resposta
- View — Templates ERB com Tailwind CSS para renderização das telas

3.2 Infraestrutura Docker

A aplicação roda em dois containers orquestrados via Docker Compose:

- Container da aplicação Rails (Ruby + Puma como servidor HTTP)
- Container do PostgreSQL

Comando para subir o ambiente completo:

```
docker compose up --build
```

3.3 Sistema de Autorização (RBAC)

O controle de acesso foi implementado manualmente com quatro tabelas dedicadas no banco:

Tabela	Descrição
papeis	Papéis disponíveis: administrador, colaborador, morador
permissoes	Permissões granulares (gerenciar_blocos, ver_chamados, etc.)
usuario_papeis	Junção: liga um usuário a um ou mais papéis
papel_permissoes	Junção: liga um papel às suas permissões

O ApplicationController centraliza a autenticação (Devise) e os métodos de autorização:

```
antes_de_todas_as_actions -> authenticate_user!
apenas_administrador!                  # redireciona se não for admin
apenas_administrador_ou_colaborador!   # redireciona se não for admin
ou colaborador
```

O model User expõe métodos helpers utilizados em controllers e views:

```
current_user.administrador?          # true/false
current_user.colaborador?            # true/false
current_user.morador?               # true/false
current_user.tem_papel?("nome")     # verificação genérica
current_user.tem_permissao?("nome") # verifica permissão específica
```

3.4 Log de Auditoria

Todas as ações importantes do sistema são registradas automaticamente na tabela log_auditoria via callbacks after_create, after_update e before_destroy nos models User, Bloco e Chamado. O model LogAuditorium oferece o método de classe registrar(usuario, acao) que centraliza essa responsabilidade.

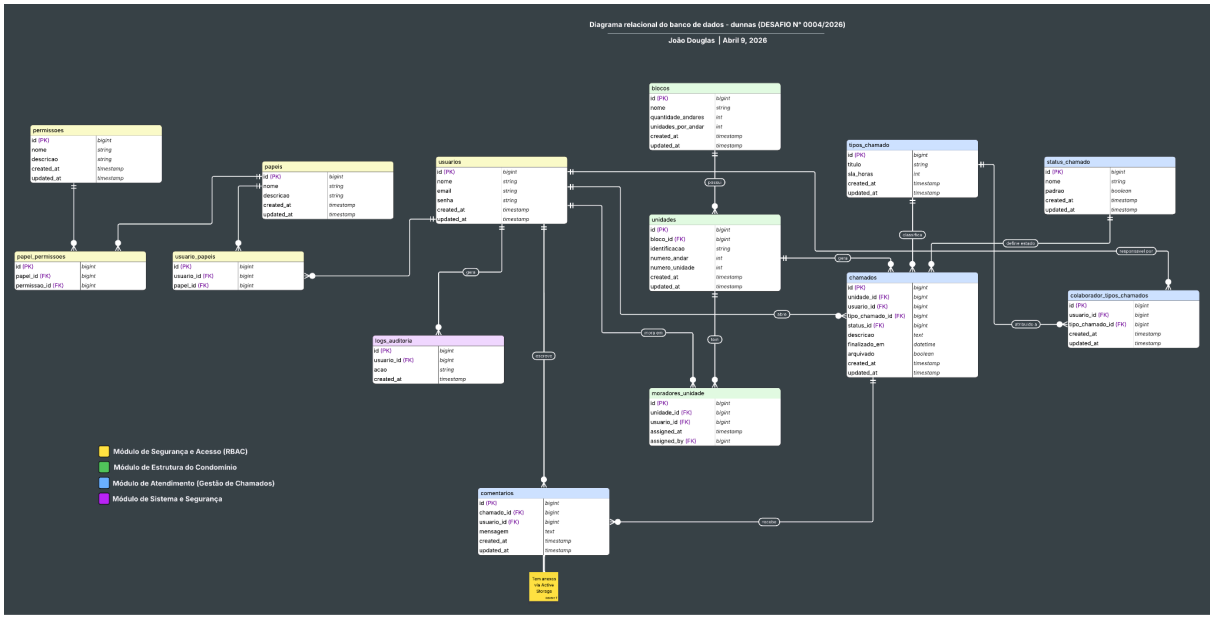
3.5 Arquivamento Automático

O job ArquivarChamadosJob é executado periodicamente via Solid Queue e arquiva automaticamente chamados com status "Concluído" há mais de 30 dias. Chamados arquivados ficam disponíveis em uma visualização separada.

4. Banco de Dados

4.1 Diagrama de Entidades

O banco é gerenciado pelo PostgreSQL e versionado via Rails Migrations. A versão atual do schema é 2026_04_14_225626.



Model / Tabela	Campos Principais	Relacionamentos
User / users	id, nome, email, encrypted_password	has_many papeis (through usuario_papeis), has_many unidades (through moradores_unidades), has_many chamados
Papel / papeis	id, nome, descricao	has_many usuarios (through usuario_papeis), has_many permissoes (through papel_permissoes)
Permissao / permissoes	id, nome, descricao	has_many papeis (through papel_permissoes)
UsuarioPapel / usuario_papeis	id, usuario_id, papel_id	belongs_to usuario, belongs_to papel
PapelPermissao / papel_permissoes	id, papel_id, permissao_id	belongs_to papel, belongs_to permissao
Bloco / blocos	id, nome, quantidade_andares, unidades_por_andar	has_many unidades (dependent: destroy)
Unidade / unidades	id, identificacao, numero_andar, numero_unidade, bloco_id	belongs_to bloco, has_many moradores (through moradores_unidades)

Model / Tabela	Campos Principais	Relacionamentos
MoradoresUnidade / moradores_unidades	id, user_id, unidade_id, assigned_at, assigned_by_id	belongs_to user, belongs_to unidade
TipoChamado / tipos_chamados	id, titulo, sla_horas	has_many chamados
StatusChamado / status_chamados	id, nome, padrao (boolean)	has_many chamados
Chamado / chamados	id, descricao, finalizado_em, arquivado, user_id, unidade_id, tipo_chamado_id, status_chamado_id	belongs_to unidade/usuario/tipo/status, has_many_attached anexos, has_many comentarios
Comentario / comentarios	id, mensagem, chamado_id, user_id	belongs_to chamado, belongs_to usuario
ColaboradorTipoChamado / colaborador_tipos_chamados	id, user_id, tipo_chamado_id	belongs_to user, belongs_to tipo_chamado
LogAuditorium / log_auditoria	id, acao (text), usuario_id, created_at	belongs_to usuario (optional)

4.2 Permissões Granulares do Sistema

O seed cria as seguintes permissões e as atribui automaticamente aos papéis:

Permissão	Administrador	Colaborador	Morador
gerenciar_blocos	Sim	Não	Não
gerenciar_usuarios	Sim	Não	Não
gerenciar_tipos_chamado	Sim	Não	Não
gerenciar_status_chamado	Sim	Não	Não
ver_chamados	Sim	Sim	Sim
atualizar_status_chamado	Sim	Sim	Não
deletar_chamados	Sim	Não	Não
ver_auditoria	Sim	Não	Não

4.3 Callbacks Importantes

Model	Callback	Ação Executada
Bloco	after_create	gerar_unidades — cria todas as unidades automaticamente
Bloco	after_update	regenerar_unidades — sincroniza unidades se estrutura mudou

Model	Callback	Ação Executada
Chamado	before_create	definir_status_padrao — atribui StatusChamado com padrao: true
Chamado	before_update	registrar_finalizacao — preenche finalizado_em ao concluir
Chamado	after_create	log_criacao — registra abertura no log de auditoria
Chamado	after_update	log_atualizacao — registra mudanças no log de auditoria
User	after_create	log_criacao — registra criação do usuário
User	after_update	log_atualizacao — registra alterações (exceto senha em claro)
Bloco	before_destroy	log_remocao — registra remoção no log de auditoria

Identificação das unidades gerada pelo método gerar_unidades:

- Andar 1, unidade 1 → identificacao = "101"
- Andar 1, unidade 2 → identificacao = "102"
- Andar 2, unidade 1 → identificacao = "201"

5. Rotas da Aplicação

Recurso / Rota	Método	Acesso
/ (root)	GET	Redireciona para /users/sign_in
/dashboard	GET	Todos os perfis (dashboard personalizado)
/chamados	GET/POST	Admin, colaborador, morador (escopo por perfil)
/chamados/:id/comentarios	POST	Admin, colaborador, morador (com restrições)
/chamados/:id/remover_anexo	DELETE	Admin e morador (dono do chamado)
/blocos	CRUD	Somente administrador
/usuarios	CRUD	Somente administrador
/usuarios/:id/vincular_unidade, desvincular_unidade, vincular_tipo_chamado, desvincular_tipo_chamado	POST/DELETE	Somente administrador
/tipos_chamado	CRUD	Somente administrador
/status_chamados	CRUD	Somente administrador
/unidades	GET	Admin e colaborador
/logs_auditoria	GET	Somente administrador
/users/sign_in	GET/POST	Público — página de login Devise
/users/password	GET/POST	Público — recuperação de senha Devise

6. Perfis de Usuário e Permissões

6.1 Administrador

Acesso total ao sistema. Responsável pela configuração inicial e gestão completa da estrutura.

- Cria e gerencia usuários (nenhum cadastro público — apenas admin cria contas)
- Cadastra blocos e vincula moradores a unidades
- Define tipos de chamado (com SLA em horas) e status de chamado
- Vê todos os chamados com filtros completos (bloco, tipo, status, período)
- Acessa o log de auditoria completo
- Pode arquivar e gerenciar chamados de qualquer unidade

6.2 Colaborador

Responsável pela gestão operacional dos chamados. Pode ser vinculado a tipos de chamado específicos.

- Vê chamados dos seus tipos de chamado responsáveis e das suas unidades vinculadas
- Atualiza status de chamados (de Aberto → Em andamento → Concluído)
- Comenta em qualquer chamado dentro do seu escopo
- Não pode abrir chamados nem acessar configurações do sistema

6.3 Morador

Usuário final do sistema. Interage apenas com chamados das suas unidades.

- Abre chamados selecionando uma das suas unidades vinculadas
- Vê apenas chamados das suas próprias unidades
- Comenta somente nos chamados das suas unidades
- Não pode alterar status de chamados

6.4 Tabela Resumida de Acesso

Funcionalidade	Admin	Colaborador	Morador
Criar usuários	Sim	Não	Não
Cadastrar blocos	Sim	Não	Não
Vincular morador a unidade	Sim	Não	Não
Cadastrar tipos de chamado	Sim	Não	Não
Cadastrar status de chamado	Sim	Não	Não

Funcionalidade	Admin	Colaborador	Morador
Ver log de auditoria	Sim	Não	Não
Ver todos os chamados	Sim	Sim*	Não
Abrir chamado	Não	Não	Sim
Alterar status de chamado	Sim	Sim*	Não
Comentar em qualquer chamado	Sim	Sim*	Não
Comentar nos próprios chamados	Não	Não	Sim
Ver chamados arquivados	Sim	Sim*	Não
Remover anexo de chamado	Não	Não	Sim
Remover chamado	Sim	Não	Não

** Escopo restrito ao tipo de chamado ou unidades vinculadas ao colaborador*

7. Como Rodar o Projeto

7.1 Pré-requisitos

- Docker instalado
- Docker Compose instalado
- Git instalado

7.2 Passo a Passo

Passo 1 — Clonar o repositório:

```
git clone <https://github.com/joaodouglasdantas/dunnas_CAPP.git>
cd capp
```

Passo 2 — Configurar variáveis de ambiente:

```
cp .env.example .env
```

Edite o arquivo .env com as configurações de banco e chaves de acesso ao S3 (se necessário).

Passo 3 — Subir os containers:

```
docker compose up --build
```

Passo 4 — Em outro terminal, criar e migrar o banco:

```
docker compose exec web rails db:create db:migrate db:seed
```

Passo 5 — Acessar no navegador:

http://localhost:3000

Ou acessar a URL de produção:

https://dunnas-capp.onrender.com

⚠️ **Comportamento do Render no plano gratuito:**

O sistema está hospedado no Render (plano gratuito) e entra em modo de hibernação após 15 minutos sem uso. Na primeira tentativa de acesso do dia, aguarde entre 30 e 60 segundos para o servidor acordar.

7.3 Credenciais Iniciais (Seed)

Campo	Valor
E-mail	admin@capp.com
Senha	admin123
Papel	Administrador

7.4 Dados Pré-configurados pelo Seed

Além do usuário administrador, o seed cria automaticamente:

- 3 papéis: administrador, colaborador, morador
- 8 permissões granulares atribuídas a cada papel
- 3 status de chamado: Aberto (padrão), Em andamento, Concluído
- 3 tipos de chamado: Manutenção (SLA 24h), Segurança (SLA 2h), Limpeza (SLA 8h)

7.5 Rodando os Testes

```
docker compose exec web rails test
```

8. Testes

8.1 Metodologia — TDD

O projeto foi desenvolvido seguindo TDD (Test-Driven Development) com o ciclo Red → Green aplicado em todas as 7 etapas:

- Red — Escrever o teste antes da implementação e confirmar que ele falha pelo motivo correto
- Green — Implementar o código mínimo necessário para o teste passar
- Refactor — Melhorar o código mantendo todos os testes verdes

Um aprendizado importante durante o projeto foi a diferença entre os tipos de falha no Minitest:

- F (Failure) — o teste rodou, mas o resultado foi diferente do esperado
- E (Error) — o teste nem conseguiu rodar por falta de código implementado

8.2 Tipos de Teste

- Testes de model (test/models/) — validações, callbacks, regras de negócio
- Testes de controller (test/controllers/) — fluxos HTTP, autorização, redirecionamentos
- Testes de job (test/jobs/) — ArquivarChamadosJob

8.3 Principais Casos de Teste

Arquivo de Teste	Casos Cobertos
user_test.rb	Nome obrigatório, email obrigatório, métodos de papel funcionam corretamente
bloco_test.rb	Validações de campos, geração automática de unidades após criação
status_chamado_test.rb	Apenas um status pode ser marcado como padrão (unicidade)
chamado_test.rb	Recebe status padrão ao ser criado, finalizado_em preenchido ao concluir
comentario_test.rb	Admin e colaborador comentam em qualquer chamado; morador só nos seus
moradores_unidade_test.rb	Vínculo entre morador e unidade
blocos_controller_test.rb	Admin acessa, não-admin é redirecionado
chamados_controller_test.rb	Morador vê só seus chamados, admin vê todos, filtros funcionam
arquivar_chamados_job_test.rb	Chamados concluídos há +30 dias são arquivados corretamente

8.4 Fixtures

Os testes utilizam fixtures (test/fixtures/) para preparar dados consistentes. Um aprendizado chave foi que o usuário de teste deve ter o papel correto para o cenário — usar um colaborador para testar restrição de morador falharia, pois colaboradores têm mais permissões.

9. Design e Interface

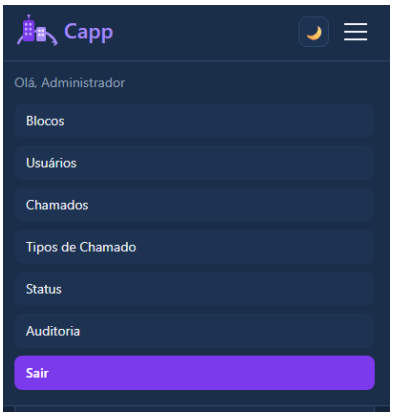
9.1 Paleta de Cores do Sistema

Uso	Cor (Hex)	Onde Aparece
Fundo principal	#152338	Fundo das páginas
Cards e tabelas	#1a2e4a	Fundo de tabelas, cards, painéis
Bordas	#2d4a6b	Bordas sutis entre elementos
Destaque / Cor primária	#7c3aed	Botões de ação, links, elementos interativos
Hover de botões	roxo escuro	Feedback visual ao passar o mouse
Texto principal	branco	Toda a tipografia sobre fundo escuro
Texto secundário	cinza claro	Labels, descrições, metadados
Badgets de status	variado	Verde (concluído), amarelo (andamento), cinza (aberto)
Botão editar	roxo	Ações de edição em tabelas
Botão remover	vermelho	Ações destrutivas em tabelas

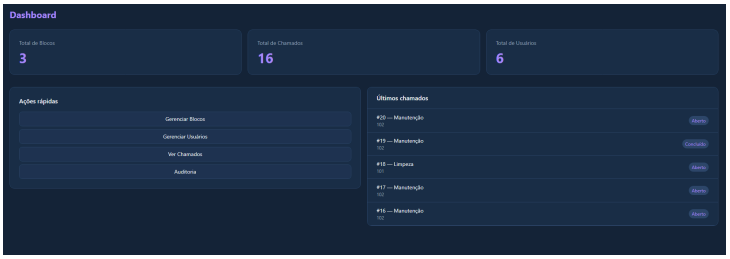
9.2 Componentes de Interface

- Navbar fixa superior com logo clicável e navegação adaptada por perfil

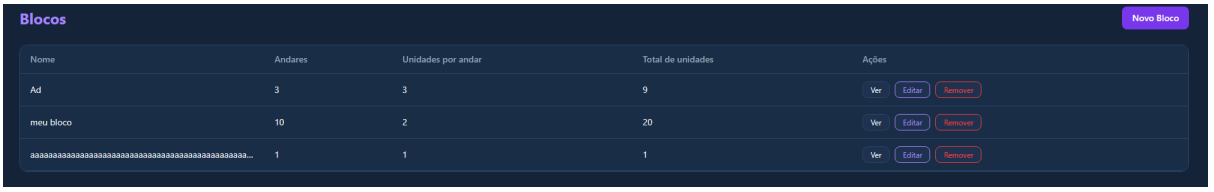
- Menu sanduíche responsivo que expande verticalmente em dispositivos móveis



- Dashboard com cards de métricas (totais de blocos, chamados, usuários) e números em destaque roxo



- Tabelas com fundo #1a2e4a, cabeçalhos em cinza, linhas com hover sutil


Blocos
Novo Bloco
Nome
Andares
Unidades por andar
Total de unidades
Ações
Ad
3
3
9
Ver
Editar
Remover
meu bloco
10
2
20
Ver
Editar
Remover
aaaa

- Variáveis CSS definidas no arquivo app/assets/tailwind/application.css

9.3 Responsividade

Todas as telas foram desenvolvidas com responsividade “completa” para desktop e mobile. A navbar colapsa em menu hambúrguer em telas menores. Os cards de dashboard reorganizam em coluna única. Tabelas possuem scroll horizontal quando necessário.

10. Estrutura de Pastas Relevantes

```
capp/
├── app/
│   ├── controllers/
│   │   ├── application_controller.rb     # autenticação + autorização
base
│   │   ├── dashboard_controller.rb       # dashboard por perfil
│   │   ├── chamados_controller.rb        # CRUD + filtros + escopo
por perfil
│   │   ├── blocos_controller.rb          # apenas admin
│   │   ├── comentarios_controller.rb     # criação de comentários
│   │   ├── usuarios_controller.rb        # apenas admin
│   │   ├── logs_auditoria_controller.rb  # apenas admin
│   │   └── admin_reset_controller.rb     # rota de reset
administrativo
│   ├── models/
│   │   ├── user.rb                     # RBAC helpers + callbacks de
auditoria
│   │   ├── chamado.rb                  # callbacks de status e
auditoria
│   │   ├── bloco.rb                    # gerar/regenerar unidades
│   │   ├── log_auditorium.rb           # .registrar(usuario, acao)
│   │   └── colaborador_tipo_chamado.rb  # vínculo colaborador ↔ tipo
│   ├── jobs/
│   │   └── arquivar_chamados_job.rb    # arquivamento automático
│   ├── views/
│   │   ├── layouts/application.html.erb # navbar + tema escuro
│   │   ├── dashboard/                  # dashboards por perfil
│   │   └── chamados/                   # listagem + formulário + show
│   └── assets/tailwind/application.css  # variáveis CSS do tema
├── db/
│   ├── migrate/                        # 17 migrations versionadas
│   ├── schema.rb                       # versão 2026_04_14_225626
│   └── seeds.rb                        # papéis, permissões, status,
tipos, admin
├── test/
│   ├── models/                         # testes de model (TDD)
│   ├── controllers/                    # testes de controller
│   └── jobs/                           # testes de jobs
├── config/
│   ├── routes.rb                       # todas as rotas da aplicação
│   ├── application.rb                  # pt-BR + Brasilia timezone
│   └── initializers/
│       ├── aws_r2.rb                   # config do storage
│       └── scheduler.rb                # agendamento do job de
arquivamento
├── docker-compose.yml                  # orquestração de containers
├── Dockerfile                          # imagem da aplicação Rails
```

```
├── render.yaml                          # configuração de deploy no
Render
└── Gemfile                              # dependências Ruby
```

11. Histórico de Desenvolvimento (7 Etapas)

Etapa 1 — Autenticação e Controle de Acesso (RBAC)

Instalação do Devise e criação do modelo RBAC manual com as tabelas papeis, permissoes, usuario_papeis e papel_permissoes. Desabilitação do cadastro público (somente admin cria contas). Adição dos helpers administrador?, colaborador?, morador? e tem_permissao? no model User.

Etapa 2 — Estrutura do Condomínio

Implementação dos models Bloco, Unidade e MoradoresUnidade. O callback after_create gera automaticamente todas as unidades no padrão 101, 102, 201, etc. Foi a primeira etapa em que o ciclo Red → Green foi aplicado de forma pura.

Etapa 3 — Tipos de Chamado e Status

Criação dos models TipoChamado (com sla_horas) e StatusChamado (com campo padrao para garantir unicidade). O diagrama do banco foi atualizado para incluir o campo padrao, demonstrando que o diagrama é um documento vivo.

Etapa 4 — Chamados e Permissões

Implementação do model Chamado com callbacks before_create (status padrão automático) e before_update (registro da data de finalização). Aprendizado sobre a diferença entre erros F e E no Minitest.

Etapa 5 — Comentários e Histórico

Implementação dos models Comentario e Anexo. O método pode_comentar? centraliza as regras de negócio — admin e colaborador podem comentar em qualquer chamado; moradores apenas nos chamados das suas unidades. Lição importante sobre fixtures corretas em testes.

Etapa 6 — Views e Controllers

Construção completa da camada de apresentação. Dashboard personalizado por perfil. ChamadosController com escopos diferenciados. Seeds com dados iniciais. Correção de vários bugs: cadastro público aberto, fuso UTC, remoção de usuários com registros vinculados, redirecionamentos incorretos.

Etapa 7 — Design e Frontend

Implementação do tema escuro/claro com paleta tons de roxo inspirada na Dunnas (#152338, #1a2e4a, #7c3aed). Navbar fixa e responsiva. Dashboard com cards de métricas. Filtros de chamados estilizados. Todas as views refatoradas com Tailwind CSS. Adição de funcionalidades extras: log de auditoria, arquivamento automático, upload de anexos, vinculação de colaboradores a tipos.

12. Glossário

Termo	Definição
RBAC	Role-Based Access Control — controle de acesso baseado em papéis
SLA	Service Level Agreement — prazo máximo em horas para resolver um chamado
TDD	Test-Driven Development — desenvolvimento orientado a testes (Red → Green)
MVC	Model-View-Controller — padrão arquitetural do Rails
Devise	Gem para autenticação Rails (login, logout, recuperação de senha, BCrypt)
BCrypt	Algoritmo de hash irreversível para armazenar senhas com segurança
Migration	Arquivo Ruby que versiona alterações no esquema do banco de dados
Seed	Script (db/seeds.rb) que popula o banco com dados iniciais de configuração
Callback	Gancho do Rails que executa lógica automática em eventos do ciclo de vida
Fixture	Dados de teste pré-definidos carregados automaticamente pelos testes
Active Storage	Módulo nativo do Rails para upload e gerenciamento de arquivos
Turbo Rails	Parte do Hotwire — acelera navegação e atualizações parciais de página
Solid Queue	Sistema de fila de jobs baseado em banco de dados (Rails 8)
Propshaft	Asset pipeline moderno do Rails 8, substituto do Sprockets
Bloco	Prédio do condomínio; possui andares e gera unidades automaticamente
Unidade	Apartamento dentro de um bloco, identificado por código (ex: 101)
Chamado	Solicitação registrada por um morador para reportar um problema
Colaborador	Perfil que gerencia chamados; pode ser vinculado a tipos específicos
Arquivado	Chamado concluído há mais de 30 dias, movido para histórico separado
Log de Auditoria	Registro imutável de todas as ações relevantes no sistema
Cloudflare R2	Serviço de armazenamento de arquivos em nuvem compatível com a API S3, usado para guardar os anexos em produção

13. Notas e Pontos de Expansão

13.1 Decisões Técnicas

- O RBAC foi implementado manualmente (sem CanCanCan) para controle total e aprendizado aprofundado
- O cadastro público foi desabilitado — a criação de contas é exclusivamente via administrador
- O campo padrao no StatusChamado não estava no diagrama original, mas foi necessário para a regra de negócio — o diagrama foi atualizado, demonstrando que é um documento vivo
- O fuso horário foi configurado para America/Sao_Paulo (Brasília) — sem exibição em UTC
- A remoção de usuários com registros vinculados foi tratada com dependent: destroy nos relacionamentos para evitar violações de chave estrangeira
- Colaboradores possuem uma tabela dedicada (colaborador_tipos_chamados) para definir seu escopo de chamados por tipo
- O upload de anexos usa Active Storage nativo do Rails com suporte configurado para Cloudflare R2
- Uma rota administrativa com chave secreta foi criada para permitir o reset dos dados diretamente pelo painel do Render, sem necessidade de acesso ao terminal

```
/admin/reset?token=umasenhaforte123
```

13.2 Pontos de Expansão Futura

- Implementar uma lógica completa de verificação de permissões granulares (tabelas criadas, lógica expandível)
- Notificações por e-mail ao abrir ou atualizar chamados (infrastructure já existe com Devise mailer)
- Relatórios e exportação de chamados em CSV ou PDF
- API REST para consumo por aplicativos mobile
- Painel de métricas com gráficos de chamados por tipo, status e período
- Notificações em tempo real usando Turbo Streams + Action Cable